Self-Checking Testbench Example

```
module basic_selfchecking_testbench();
  logic [3:0] stim1, stim2, result;
  logic carry;
  // DUT for this example is a 4-bit adder with carry out
  full_adder4 iNAME(.clk(clk), .rst_n(rst_n), .in1(stim1), .in2(stim2), .S(result), Cout(carry));

  initial begin
    // Testcase 1: 3 + 12 = 15, check normal operation
    stim1 = 4'd3;  stim2 = 4'd12;
    #5;
    if ( (result !== 4'd15) || (carry !== 1'b0) ) begin
      $display("in1=%d, in2=%d, expected S=%d Cout=%d but got S=%d Cout=%d\n",
               stim1, stim2, 4'd15, 1'b0, result, carry);
      $stop();
    end

    // Testcase 2: 14 + 12 = 26 -> Cout=1, S=10, check overflow occurs and sum preserves lower bits
    stim1 = 4'hE;  stim2 = 4'hC;
    #5;
    if ( (result !== 4'hA) || (carry !== 1'b1) ) begin
      $display("in1=%d, in2=%d, expected S=%d Cout=%d but got S=%d Cout=%d\n",
               stim1, stim2, 4'd15, 1'b0, result, carry);
      $stop();
    end

    ... You would have more cases here

    $display("All tests passed!\n"); $stop();
  end
endmodule
```

check result and carry values against expected

print out values using formatted string for easier debugging

don't continue simulation if we failed a case

doesn't matter if we specify with decimal or hex

print a happy message if all is good :)